

VE Governance - Updates *Aragon*

HALBORN

VE Governance - Updates - Aragon

Prepared by:  HALBORN

Last Updated 06/26/2025

Date of Engagement: May 19th, 2025 - May 30th, 2025

Summary

88% ⓘ OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED

ALL FINDINGS	CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL
8	1	0	3	0	4

TABLE OF CONTENTS

- 1. Summary
- 2. Introduction
- 3. Assessment summary
- 4. Test approach and methodology
- 5. Warm-up period evaluation in linearincreasingcurve
- 6. Risk methodology
- 7. Scope
- 8. Assessment summary & findings overview
- 9. Findings & Tech Details
 - 9.1 Unauthorized initialization of critical contract parameter
 - 9.2 Incorrect warm-up period evaluation for historical timestamps
 - 9.3 Incorrect historical voting power calculation for merged tokens
 - 9.4 Inactive gauge votes not properly handled during updates
 - 9.5 Inconsistent decimal handling in voting power updates
 - 9.6 Inefficient timestamp calculation in checkpoint resolution
 - 9.7 Inconsistent maxtime boundary handling in voting power calculation
 - 9.8 Misleading event name for auto-delegation state change

1. Summary

2. Introduction

Aragon engaged our security analysis team to perform a comprehensive security assessment of their smart contract ecosystem. The primary objective was to thoroughly evaluate the security architecture of the smart contracts to identify vulnerabilities, assess existing security measures, and provide actionable recommendations to enhance the security and operational effectiveness of their smart contract framework. Our assessment was strictly limited to the provided smart contracts, ensuring a focused and exhaustive analysis of their security features.

3. Assessment Summary

Our engagement with Aragon spanned a 10-day period, during which we assigned a full-time security engineer with extensive experience in blockchain security, advanced penetration testing skills, and deep knowledge of various blockchain protocols. The objectives of this assessment were to:

- Verify the correct functionality of the smart contract operations.
- Identify potential security vulnerabilities within the smart contracts.
- Provide recommendations to improve the security and efficiency of the smart contracts.

4. Test Approach And Methodology

Our testing strategy combined manual and automated techniques to ensure a comprehensive evaluation. Manual testing was essential for detecting logical and implementation flaws, while automated testing provided broad code coverage and rapid identification of common vulnerabilities. The testing process included:

- A detailed review of the smart contracts' architecture and intended functionality.
- Comprehensive manual code reviews and walkthroughs.
- Functional and connectivity analysis using tools such as Solgraph.
- Customized script-based manual testing and testnet deployment using Foundry.

This executive summary highlights the key findings and recommendations from our security assessment of the Aragon smart contract ecosystem. By addressing the identified issues and implementing the recommended improvements, Aragon can significantly enhance the security, reliability, and trustworthiness of its smart contract platform.

5. Warm-Up Period Evaluation In LinearIncreasingCurve

The `LinearIncreasingCurve` contract's `_isWarm` function includes a conditional check based on `_originalPoint.checkpointTs > _originalPoint.writtenTs` to preserve backward compatibility with the previous implementation. This check ensures that voting power calculations remain consistent for existing tokens both before and after the upgrade.

The previous implementation used a different approach, evaluating the warm-up period based on `writtenTs` rather than `checkpointTs`. Moreover, it always relied on `block.timestamp` for evaluation, which prevented accurate calculation of historical voting power. Additionally, the old code exhibited inconsistent behavior between binary search (which used `checkpointTs`) and warm-up evaluation (which used `writtenTs`).

The current implementation intentionally preserves this behavior to ensure existing tokens continue to operate as expected. This is critical because:

1. The warm-up period evaluation must yield identical results for the same token before and after the upgrade.
2. Altering the logic would cause discrepancies in voting power calculations, potentially disrupting existing functionality.
3. Edge cases, especially those involving very short warm-up periods (as brief as one second), must be handled correctly to avoid underflow errors.

The conditional check in `_isWarm` is necessary because:

1. For legacy points where `checkpointTs > writtenTs`, the evaluation uses `writtenTs + warmupPeriod` to maintain the original behavior.
2. For new points where `checkpointTs == writtenTs`, it uses `locked.start + warmupPeriod`.
3. This approach ensures that warm-up period evaluation remains consistent with the previous implementation.

Although the old implementation's design differs from the current one, maintaining backward compatibility is a deliberate design choice rather than a flaw. Changing this behavior would break existing functionality for deployed tokens. Given the constraints of supporting existing contracts, the current implementation offers the most stable solution.

For future implementations, consider the following improvements:

1. Consistently using `checkpointTs` for warm-up period evaluation.
2. Adding a timestamp parameter to `_isWarm` to support historical calculations.
3. Ensuring consistent behavior between binary search and warm-up evaluation.

6. RISK METHODOLOGY

Every vulnerability and issue observed by Halborn is ranked based on **two sets of Metrics** and a **Severity Coefficient**. This system is inspired by the industry standard Common Vulnerability Scoring System.

The two **Metric sets** are: **Exploitability** and **Impact**. **Exploitability** captures the ease and technical means by which vulnerabilities can be exploited and **Impact** describes the consequences of a successful exploit.

The **Severity Coefficients** is designed to further refine the accuracy of the ranking with two factors: **Reversibility** and **Scope**. These capture the impact of the vulnerability on the environment as well as the number of users and smart contracts affected.

The final score is a value between 0-10 rounded up to 1 decimal place and 10 corresponding to the highest security risk. This provides an objective and accurate rating of the severity of security vulnerabilities in smart contracts.

The system is designed to assist in identifying and prioritizing vulnerabilities based on their level of risk to address the most critical issues in a timely manner.

6.1 EXPLOITABILITY

ATTACK ORIGIN (AO):

Captures whether the attack requires compromising a specific account.

ATTACK COST (AC):

Captures the cost of exploiting the vulnerability incurred by the attacker relative to sending a single transaction on the relevant blockchain. Includes but is not limited to financial and computational cost.

ATTACK COMPLEXITY (AX):

Describes the conditions beyond the attacker’s control that must exist in order to exploit the vulnerability. Includes but is not limited to macro situation, available third-party liquidity and regulatory challenges.

METRICS:

EXPLOITABILITY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Origin (AO)	Arbitrary (AO:A) Specific (AO:S)	1 0.2
Attack Cost (AC)	Low (AC:L) Medium (AC:M) High (AC:H)	1 0.67 0.33

EXPLOITABILITY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Complexity (AX)	Low (AX:L) Medium (AX:M) High (AX:H)	1 0.67 0.33

Exploitability E is calculated using the following formula:

$$E = \prod m_e$$

6.2 IMPACT

CONFIDENTIALITY (C):

Measures the impact to the confidentiality of the information resources managed by the contract due to a successfully exploited vulnerability. Confidentiality refers to limiting access to authorized users only.

INTEGRITY (I):

Measures the impact to integrity of a successfully exploited vulnerability. Integrity refers to the trustworthiness and veracity of data stored and/or processed on-chain. Integrity impact directly affecting Deposit or Yield records is excluded.

AVAILABILITY (A):

Measures the impact to the availability of the impacted component resulting from a successfully exploited vulnerability. This metric refers to smart contract features and functionality, not state. Availability impact directly affecting Deposit or Yield is excluded.

DEPOSIT (D):

Measures the impact to the deposits made to the contract by either users or owners.

YIELD (Y):

Measures the impact to the yield generated by the contract for either users or owners.

METRICS:

IMPACT METRIC (M_I)	METRIC VALUE	NUMERICAL VALUE
Confidentiality (C)	None (I:N) Low (I:L) Medium (I:M) High (I:H) Critical (I:C)	0 0.25 0.5 0.75 1

Impact Metric (M_I)	Metric Value	Numerical Value
Integrity (I)	None (I:N)	0
	Low (I:L)	0.25
	Medium (I:M)	0.5
	High (I:H)	0.75
	Critical (I:C)	1
Availability (A)	None (A:N)	0
	Low (A:L)	0.25
	Medium (A:M)	0.5
	High (A:H)	0.75
	Critical (A:C)	1
Deposit (D)	None (D:N)	0
	Low (D:L)	0.25
	Medium (D:M)	0.5
	High (D:H)	0.75
	Critical (D:C)	1
Yield (Y)	None (Y:N)	0
	Low (Y:L)	0.25
	Medium (Y:M)	0.5
	High (Y:H)	0.75
	Critical (Y:C)	1

Impact I is calculated using the following formula:

$$I = \text{max}(m_I) + \frac{\sum m_I - \text{max}(m_I)}{4}$$

6.3 SEVERITY COEFFICIENT

REVERSIBILITY (R):

Describes the share of the exploited vulnerability effects that can be reversed. For upgradeable contracts, assume the contract private key is available.

SCOPE (S):

Captures whether a vulnerability in one vulnerable contract impacts resources in other contracts.

METRICS:

Severity Coefficient (C)	Coefficient Value	Numerical Value
Reversibility (r)	None (R:N)	1
	Partial (R:P)	0.5
	Full (R:F)	0.25
Scope (s)	Changed (S:C)	1.25
	Unchanged (S:U)	1

Severity Coefficient *C* is obtained by the following product:

$$C = rs$$

The Vulnerability Severity Score *S* is obtained by:

$$S = \min(10, EIC * 10)$$

The score is rounded up to 1 decimal places.

SEVERITY	SCORE VALUE RANGE
Critical	9 - 10
High	7 - 8.9
Medium	4.5 - 6.9
Low	2 - 4.4
Informational	0 - 1.9

7. SCOPE

FILES AND REPOSITORY
(a) Repository: ve-governance (b) Assessed Commit ID: 15e61fd (c) Items in scope: - EscrowVotesAdapter.sol - LinearIncreasingCurve.sol - VotingEscrowIncreasing_v1_2_0.sol - GaugesDaoFactory_v1_2_0.sol - GaugesDaoFactory_v1_3_0.sol - GaugesDaoFactory.sol - GaugesDaoFactory_v1_1_0.sol - UpgradeFactory_v1_0_0_v1_3_0.sol - AddressGaugeVoter.sol - GaugeVoterSetup_v1_2_0.sol - GaugeVoterSetup_v1_3_0.sol - GaugeVoterSetup_v1_1_0.sol - LinearIncreasingCurveNoSupply.sol - Clock_v1_2_0.sol - Lock_v1_2_0.sol
Out-of-Scope:
REMEDIATION COMMIT ID:
https://github.com/aragon/ve-governance/pull/136 https://github.com/aragon/ve-governance/pull/138 https://github.com/aragon/ve-governance/pull/135/files https://github.com/aragon/ve-governance/pull/137/files
Out-of-Scope: New features/implementations after the remediation commit IDs.

8. ASSESSMENT SUMMARY & FINDINGS OVERVIEW

CRITICAL	HIGH	MEDIUM	LOW
1	0	3	0

INFORMATIONAL

4

SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
UNAUTHORIZED INITIALIZATION OF CRITICAL CONTRACT PARAMETER	CRITICAL	SOLVED - 06/05/2025
INCORRECT WARM-UP PERIOD EVALUATION FOR HISTORICAL TIMESTAMPS	MEDIUM	NOT APPLICABLE
INCORRECT HISTORICAL VOTING POWER CALCULATION FOR MERGED TOKENS	MEDIUM	SOLVED - 06/23/2025
INACTIVE GAUGE VOTES NOT PROPERLY HANDLED DURING UPDATES	MEDIUM	PENDING
INCONSISTENT DECIMAL HANDLING IN VOTING POWER UPDATES	INFORMATIONAL	SOLVED
INEFFICIENT TIMESTAMP CALCULATION IN CHECKPOINT RESOLUTION	INFORMATIONAL	SOLVED - 06/05/2025
INCONSISTENT MAXTIME BOUNDARY HANDLING IN VOTING POWER CALCULATION	INFORMATIONAL	NOT APPLICABLE - 06/05/2025
MISLEADING EVENT NAME FOR AUTO-DELEGATION STATE CHANGE	INFORMATIONAL	SOLVED - 06/05/2025

9. FINDINGS & TECH DETAILS

9.1 UNAUTHORIZED INITIALIZATION OF CRITICAL CONTRACT PARAMETER

// CRITICAL

Description

The `AddressGaugeVoter` contract contains a critical vulnerability in its `initializeFrom` function. This function is publicly accessible without any access control, allowing any caller to set the `ivotesAdapter` address—a crucial parameter that governs the contract's entire voting balances.

The current implementation is as follows:

```
function initializeFrom(address _ivotesAdapter) public {
    ivotesAdapter = _ivotesAdapter;
}
```

This presents a severe security risk because:

1. The `ivotesAdapter` address is used to determine voting power via `IVotes(ivotesAdapter).getVotes()`.
2. Any user can change this address to point to a malicious contract.
3. The malicious contract could return arbitrary voting power values.
4. This would enable attackers to manipulate the entire voting system.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:C/A:C/I:C/D:C/Y:C (10.0)

Recommendation

To mitigate this issue, either:

1. Remove the `initializeFrom` function entirely if it is unnecessary; or
2. Implement proper access control by adding the `auth` modifier with the `GAUGE_ADMIN_ROLE`, similar to the `setIVotesAdapter` function:

```
function initializeFrom(address _ivotesAdapter) external auth(GAUGE_ADMIN_ROLE) {
    ivotesAdapter = _ivotesAdapter;
}
```

This change will ensure that only authorized administrators can modify this critical parameter.

Remediation Comment

SOLVED: The Aragon team solved this finding.

Remediation Hash

<https://github.com/aragon/ve-governance/pull/136>

9.2 INCORRECT WARM-UP PERIOD EVALUATION FOR HISTORICAL TIMESTAMPS

// MEDIUM

Description

The `_isWarm` function in the `QuadraticIncreasingCurve` contract contains a critical flaw in its implementation. It accepts only a `TokenPoint` parameter and always uses `block.timestamp` to evaluate the warm-up period, which is incorrect when calculating voting power for historical timestamps. This issue impacts the `votingPowerAt` function, which is intended to calculate voting power at any specified timestamp. When invoked with a past timestamp, the warm-up period evaluation is inaccurate because it relies on the current block timestamp rather than the target timestamp.

The current implementation is as follows:

```
function _isWarm(TokenPoint memory _point) public view returns (bool) {  
    return block.timestamp > _point.writtenTs + warmupPeriod;  
}
```

Consequently, if `votingPowerAt` is called with a historical timestamp that falls before the warm-up period would have ended, it may still return a non-zero voting power if the current block timestamp exceeds the warm-up period.

BVSS

AO:A/AC:L/AX:L/R:N/S:C/C:N/A:N/I:M/D:N/Y:N (6.3)

Recommendation

Update the `_isWarm` function to accept a timestamp parameter and use it instead of `block.timestamp`. This change will ensure that warm-up periods are accurately evaluated for any given timestamp, whether current or historical. The function should be revised to align with the `votingPowerAt` function's capability to calculate voting power at any point in time.

Remediation Comment

NOT APPLICABLE: This code is out of scope of the audit but was acknowledged by Aragon. This will be deprecated with the new release but the team is aware that current deployed contracts until the point of the audit were incorrect.

9.3 INCORRECT HISTORICAL VOTING POWER CALCULATION FOR MERGED TOKENS

// MEDIUM

Description

The `_isWarm` function in the `LinearIncreasingCurve` contract contains a critical flaw in handling merged tokens. Currently, it returns `false` when `locked.amount` is zero, incorrectly assuming the token never existed. As a result, `votingPowerAt` always returns zero for merged tokens, even for historical timestamps prior to the merge.

This issue is further complicated by the merge process in `VotingEscrowV1_2_0`. When a token is merged, its `locked` balance is reset entirely to zero values: `_locked[_from] = LockedBalance(0, 0)`.

This creates a twofold problem:

1. The `_isWarm` function incorrectly returns `false` due to the zero amount check.
2. The `locked.start` timestamp is lost, which affects the warm-up period calculation even if the amount check is removed.

Consequently, for timestamps before the merge, the token's voting power calculation is inaccurate because:

- The warm-up period check fails due to the zero amount.
- Even if the amount check is corrected, the warm-up period remains incorrect due to the missing start timestamp.
- Historical voting power calculations do not reflect the token's actual state before the merge.

BVSS

AO:A/AC:L/AX:L/R:N/S:C/C:N/A:N/I:M/D:N/Y:N (6.3)

Recommendation

Modify the merge process to preserve the original `start` timestamp when clearing the merged token's balance. Instead of resetting both amount and start to zero, only reset the amount to zero:

```
_locked[_from] = LockedBalance(0, oldLockedFrom.start);
```

This change, combined with removing the `locked.amount == 0` check from `_isWarm`, will ensure that:

1. Historical voting power calculations remain accurate for merged tokens.
2. The warm-up period is correctly evaluated based on the token's original start time.
3. Voting power calculations accurately reflect the token's state at any given timestamp.

The system's epoch alignment should prevent edge cases that might otherwise affect future voting power calculations.

Remediation Comment

SOLVED: Warm functionality was removed and deprecated.

9.4 INACTIVE GAUGE VOTES NOT PROPERLY HANDLED DURING UPDATES

// MEDIUM

Description

The `AddressGaugeVoter` contract exhibits two related issues concerning the handling of inactive gauges. When a gauge is deactivated via `deactivateGauge`, the existing votes for that gauge remain intact. These votes continue to be counted in `epochGaugeVotes`, the voting power remains included in `epochTotalVotingPowerCast`, and users can still view their votes for inactive gauges.

During `_updateVotingPower`, the function does not verify whether gauges are active when recasting votes. It processes all existing votes from `voteData.gaugesVotedFor`, generates new vote data without checking the gauge status, and recasts votes for all gauges, including inactive ones. This behavior allows users to maintain or even increase their voting power on inactive gauges.

This results in inconsistencies where inactive gauges continue to accumulate voting power, users retain voting power on gauges that should no longer accept votes, and the system fails to accurately reflect the deactivated status of gauges.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:L/D:M/Y:N (5.6)

Recommendation

Update `_updateVotingPower` to exclude inactive gauges when recasting votes. The function should verify the gauge status before including it in the new vote data. Consider implementing a mechanism to clean up votes for deactivated gauges—either automatically upon gauge deactivation, as a separate callable function to remove inactive gauge votes, or during the next voting cycle. This approach will ensure that inactive gauges do not continue to accumulate voting power, the system accurately reflects the deactivated state of gauges, and users cannot maintain or increase their voting power on inactive gauges.

9.5 INCONSISTENT DECIMAL HANDLING IN VOTING POWER UPDATES

// INFORMATIONAL

Description

The `AddressGaugeVoter` contract exhibits inconsistency in decimal scaling during voting power updates. This issue originates from the interaction between the `_updateVotingPower` function, the `_castVote` function, and the `GaugeVote` struct.

Current process flow:

1. `_voteData.voteWeights` stores values scaled by $10e32$.
2. Within `_updateVotingPower`, these scaled values are directly used to construct `newVoteData`.
3. The `_normalizedWeight` function then applies the $10e32$ scaling factor again.
4. The `_castVote` function does not utilize the `weight` parameter from `GaugeVote`.

This issue has remained undetected because:

1. The `_getTotalWeight` function correctly sums the already scaled values (with the $10e32$ factor).
2. When `_normalizedWeight` is invoked, both the numerator and denominator include the same $10e32$ scaling factor.
3. The division within `_normalizedWeight` effectively cancels out the double scaling.
4. The `_castVote` function does not use the `weight` parameter at all.

Although this does not result in incorrect calculations, it introduces unnecessary complexity and potential risks for future maintenance.

BVSS

AO:S/AC:L/AX:L/R:N/S:U/C:N/A:N/I:H/D:N/Y:N (1.5)

Recommendation

1. Update the `_updateVotingPower` function to correctly scale down the weights when creating `newVoteData`:

```
newVoteData[i] = GaugeVote(voteData.voteWeights[gauge] / 10e32, gauge);
```

2. Simplify the `_castVote` function by removing the unused `GaugeVote` parameter and passing only the gauge address directly:

```
function _castVote(
    address _gauge,
    uint256 _epoch,
    address _account,
    uint256 _votingPower,
    uint256 _voteWeight,
    AddressVoteData storage _voteData
) internal returns (uint256)
```

These changes will:

- Ensure consistent decimal handling.
- Eliminate unnecessary parameter passing.

- Enhance code maintainability by clearly indicating which values are utilized.
- Prevent potential future issues if the weight value is ever used within `_castVote`.

Remediation Comment

SOLVED: Only the `vault` parameter recommendation was applied. The decimal scale down fix cannot be applied as per the following voting case:

```
[  
  {weight: 1, gauge: gauge1},  
  {weight: 1, gauge: gauge2}  
]
```

The stored `voteWeights` during voting will be less than `10e32` as the value will be calculated as $1 * 10e32 / 2 = 5e32$.

If during the past votes we divide with `10e32` we will essentially get `0`.

Remediation Hash

<https://github.com/aragon/ve-governance/pull/138>

9.6 INEFFICIENT TIMESTAMP CALCULATION IN CHECKPOINT RESOLUTION

// INFORMATIONAL

Description

The `resolveEpochPrevCheckpointTs` function in the `clockv1_2_0` contract employs a more complex and gas-inefficient method than necessary to calculate the previous checkpoint timestamp. The current approach adds the time until the next checkpoint and then subtracts the checkpoint interval, resulting in unnecessary computational steps.

This function can be simplified by directly utilizing the `resolveEpochPrevCheckpointElapsed` function, which already computes the elapsed time since the previous checkpoint. By subtracting this elapsed time from the current timestamp, the previous checkpoint timestamp can be obtained more straightforwardly and with improved gas efficiency.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

Replace the existing implementation with a simplified calculation that directly uses the elapsed time since the previous checkpoint. This modification will reduce gas consumption and enhance code readability while preserving the original functionality. Specifically, subtract the elapsed time from the current timestamp to determine the previous checkpoint timestamp.

Remediation Comment

SOLVED: The Aragon team solved this finding.

Remediation Hash

<https://github.com/aragon/ve-governance/pull/135/files>

9.7 INCONSISTENT MAXTIME BOUNDARY HANDLING IN VOTING POWER CALCULATION

// INFORMATIONAL

Description

The `votingPowerAt` function in the `LinearIncreasingCurve` contract performs a manual check for the maximum time boundary by calculating `timeTillMaxTime` and comparing it with `elapsed`. This approach is inconsistent with the contract's design pattern, as a dedicated `boundElapsedMaxTime` function already exists to handle this logic.

The current implementation:

1. Calculates `timeTillMaxTime` as `end - lastPoint.writtenTs`
2. Compares `elapsed` with `timeTillMaxTime`
3. Assigns `elapsed` the value of `timeTillMaxTime` if it exceeds it

This manual boundary check is redundant because the `boundElapsedMaxTime` function already provides this functionality in a more consistent and reusable manner.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

Replace the manual maxTime boundary check with the existing `boundElapsedMaxTime` function to ensure consistency and reduce code duplication. This modification will improve maintainability and minimize the risk of errors if the maxTime logic requires future updates.

Remediation Comment

NOT APPLICABLE: After multiple iterations on different changes described here

<https://gist.github.com/novaknole/d0a4507c90cdd8741253f6e591b1e600> it was agreed that the best implementation is the current one.

9.8 MISLEADING EVENT NAME FOR AUTO-DELEGATION STATE CHANGE

// INFORMATIONAL

Description

The `EscrowIVotesAdapter` contract emits an `AutoDelegationSet` event when the auto-delegation state changes. However, the event name is misleading because it does not clearly specify whether auto-delegation is being enabled or disabled. This event is emitted within the `setAutoDelegationDisabled` function, which exclusively manages disabling auto-delegation.

The current implementation is as follows:

```
function setAutoDelegationDisabled(bool _disabled) external {
    address sender = _msgSender();
    autoDelegationDisabled_[sender] = _disabled;
    emit AutoDelegationSet(sender, _disabled);
}
```

The event name `AutoDelegationSet` implies a general state change, whereas the function and parameter names clearly indicate that the operation specifically concerns the disabled state. This inconsistency may cause confusion when interpreting event logs or integrating with the contract.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

Rename the event to `AutoDelegationDisabledSet` to accurately represent that the event logs the setting of the auto-delegation disabled state. This change will align the event name with the function and parameter names, improving clarity for users and integrators regarding the state change being recorded.

Remediation Comment

SOLVED: The event was renamed to `AutoDelegationDisabledSet`.

Remediation Hash

<https://github.com/aragon/ve-governance/pull/137/files>

Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the project's integrity and addressing potential vulnerabilities introduced by code modifications.